



ASCII

Pointer



Dasar Teori Pointer

Pointer adalah sebuah variabel khusus dalam bahasa pemrograman C++ yang digunakan untuk menyimpan alamat memori dari suatu data atau variabel lain. Alamat memori ini merujuk ke lokasi di mana data tersebut disimpan di dalam memori komputer. Dengan menggunakan pointer, program dapat mengakses, memanipulasi, atau bahkan mengelola data secara langsung di tingkat memori, memberikan fleksibilitas dan efisiensi yang lebih besar dalam pengelolaan resources.

Pointer dideklarasikan dengan menambahkan tanda * (simbol asterisk) sebelum nama variabel, dan alamat memori suatu variabel dapat diperoleh menggunakan operator & (address-of). Untuk mengakses nilai yang disimpan di alamat memori yang ditunjuk oleh pointer, digunakan operator * (dereference).

Secara sederhana, pointer adalah alat yang memungkinkan programmer untuk bekerja langsung dengan memori, menjadikannya elemen penting dalam pemrograman tingkat rendah dan manajemen memori dinamis.



Memori dan Alamat

Sebelum memahami tentang pointer, sebaiknya memahami dulu tentang alamat dalam memori komputer. Lokasi pada memori komputer itu memiliki alamat dan menyimpan sebuah nilai. Alamat atau address yang dimaksud bernilai numerik (umumnya dalam bentuk hexadecimal).

Setiap variabel yang dibuat pada program akan diberi lokasi di memori komputer. Nilai dari variabel sebenarnya disimpan pada alamat yang diberikan. Untuk mengetahui dimana data disimpan, pada C++ digunakan operator &.



Memori

Indonesia (Wilayah)

Address/Alamat

Rumah No. 67 Jl. Pramuka, Kecamatan Sebulu,
Provinsi Jawa Tenggara

Variabel

rumahUcup

Value/Nilai

Ucup



Pass By Value

Secara default di C++ semua variable itu passing by value. Bukan by reference. Artinya jika kita mengirim sebuah variabel ke dalam function, atau variable lain, sebenarnya yang dikirim adalah duplikasi value nya.

Contoh Program Pass By Value:

```
// passByValue.cpp
# include <iostream>
using namespace std;

struct Address{
    string kota;
    string provinsi;
    string negara;
};

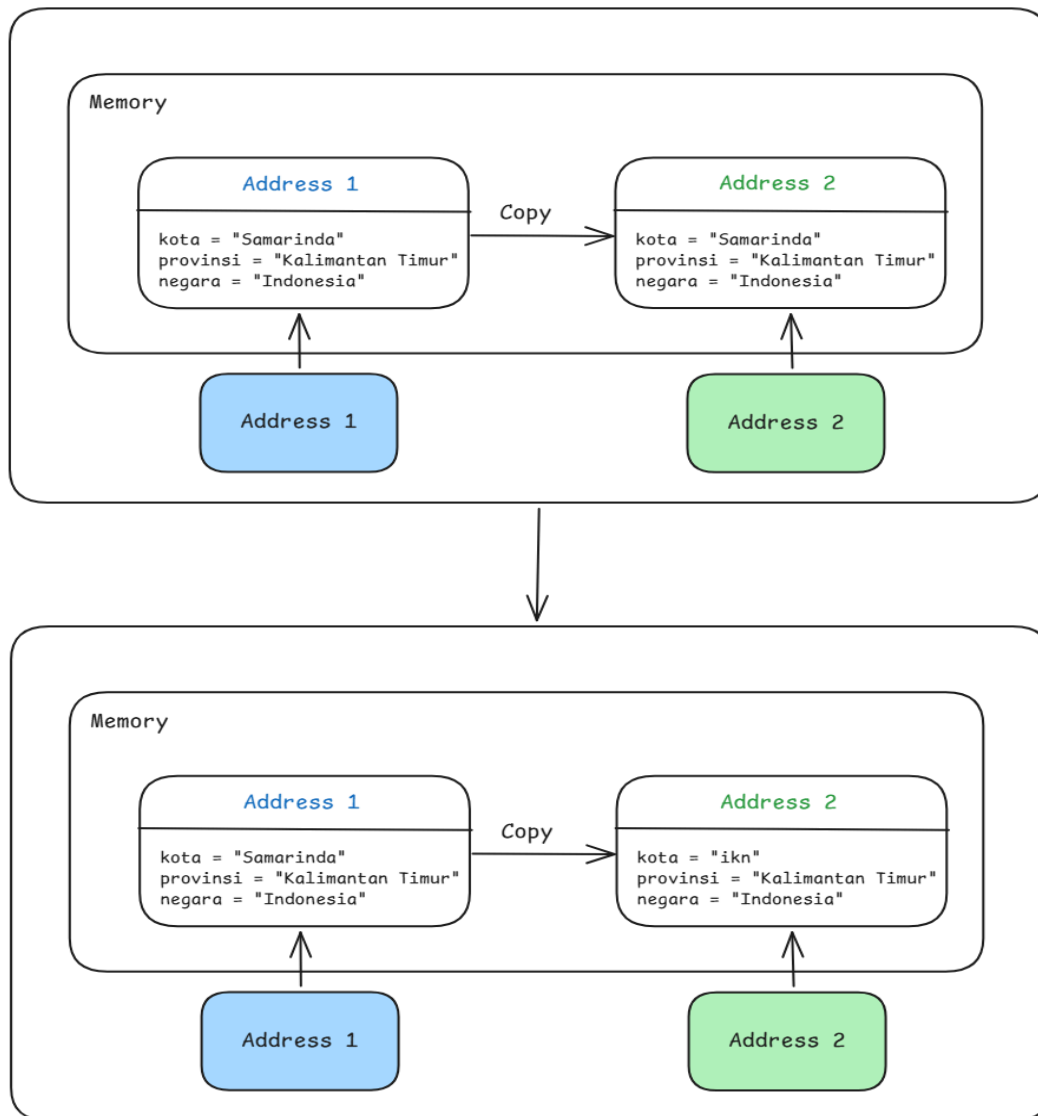
int main(){
    // deklarasi variabel dengan struct
    Address address1, address2;
    // mengisi value address1
    address1.kota = "Samarinda";
    address1.provinsi = "Kalimantan Timur";
    address1.negara = "Indonesia";
    // mengcopy value address1 ke address2
    address2 = address1;
    // mengganti property kota dari samarinda ke ikn
    address2.kota = "ikn";
    // value address1
    cout<< address1.kota<<endl;
    // value address2
    cout<< address2.kota;
    return 0;
}
```



Output :

```
→g++ passByValue.cpp ; ./a.exe  
Samarinda  
ikn
```

Visualisasi proses pengubahan alamat dengan contoh di atas



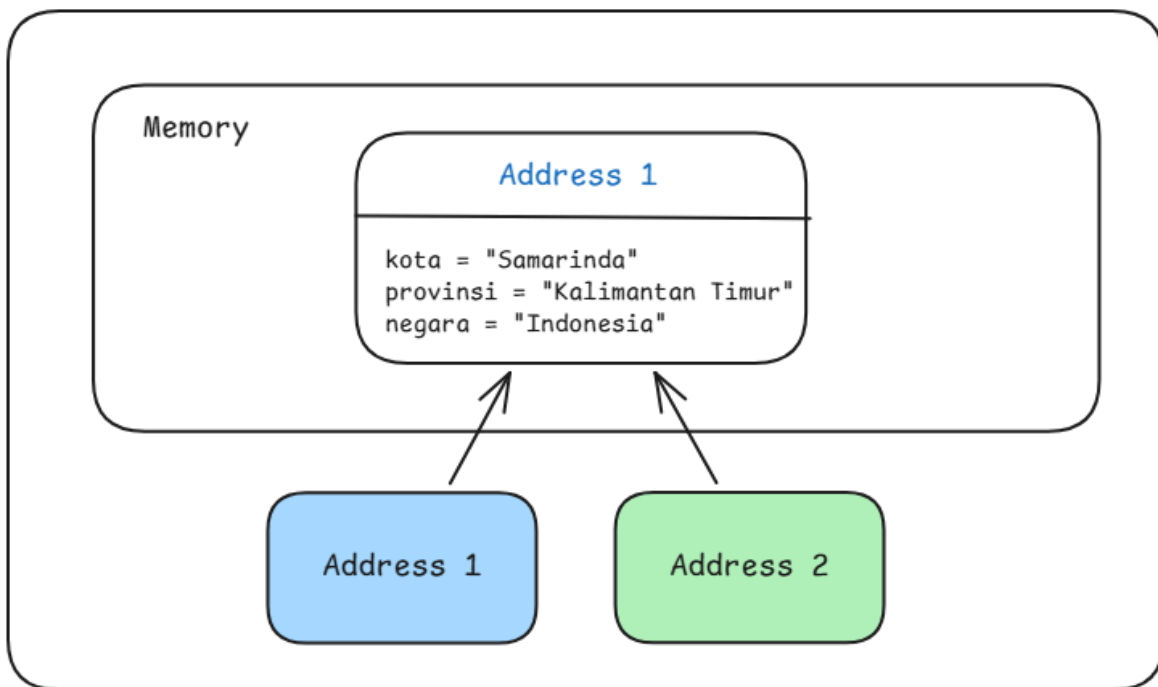
Terlihat bahwa address1 dan address2 menunjuk alamat yang berbeda



Pointer/Mengakses Pass By Reference

Pointer (penunjuk) adalah kemampuan membuat reference ke lokasi data pada memory yang sama, tanpa menduplikasi data yang sudah ada. Sederhananya, dengan kemampuan pointer, kita bisa membuat pass by reference.

Sebagai contoh, jika address1 dan address2 menunjuk ke alamat memori yang sama, maka keduanya berbagi sumber data yang identik. Konsekuensinya, jika nilai pada address2 diubah, maka address1 akan ikut berubah secara otomatis karena keduanya merujuk pada titik memori yang sama.



Jenis jenis operator dalam pointer yaitu:

- Address-of Operator (&)
- Dereference Operator (*)



Address-Of Operator (&)

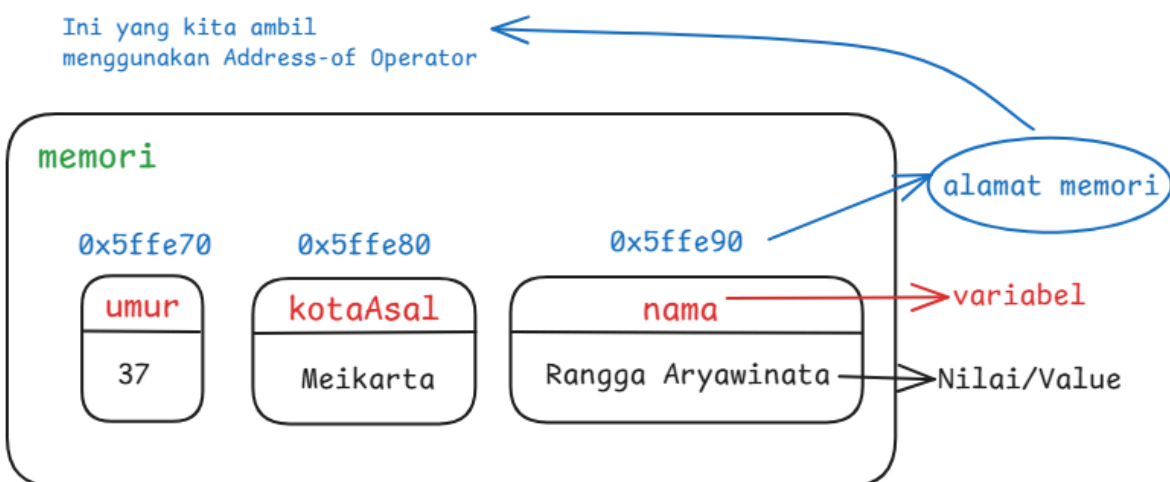
Address-of Operator (&) adalah operator yang memungkinkan kita untuk mendapatkan/melihat alamat memori yang dimiliki oleh variabel tersebut. Cara menggunakannya adalah dengan meletakkan tanda (&) di depan identitas saat pemanggilan variabel. Hal itu akan membuat compiler memberikan alamat memori, bukan isi/nilai dari memori tersebut.

Contoh:

```
#include <iostream>
using namespace std;
int main () {
    string nama = "Rangga Aryawinata";
    cout << &nama << " adalah alamatnya " << nama << endl;
    return 0;
}
```

Output:

```
→ g++ addressof.cpp ; ./a.exe
0x5ffe90 adalah alamatnya Rangga Aryawinata
```





Dereference Operator (*) Dan Membuat Pointer

Jika pada address-of operator memungkinkan kita untuk melihat alamat dari variabel yang telah kita buat, maka untuk dereference operator ini berlaku kebalikannya di mana operator ini memungkinkan kita untuk melihat nilai yang tersimpan pada alamat memori.

Sebelum itu kita akan belajar bagaimana cara membuat sebuah variabel pointer

kita sudah tau kalau operator & (ampersand) dipakai untuk mengambil alamat/address jika kita ingin memasukkan address sebagai value ke dalam Variabel maka kita memerlukan operator * (asterisk).

```
// buatPtr.cpp
#include <iostream>
using namespace std;
int main () {
    string var = "Aku Variabel";
    string *varPtr = &var;
    cout << "Hasil dari varPtr: " << varPtr << endl;
    cout << "Hasil dari *varPtr: " << *varPtr << endl;

    cout << endl << "Kesimpulannya varPtr isi nya alamatnya
var"<<endl;
    cout << "*varPtr hasilnya value dari var"<<endl;
    cout << "Jika masih bingung bisa amati output berikut"<<endl;
    cout << endl;

    cout << "Hasil/value dari var " << var << endl;
    cout << "Hasil/value dari alamat var (&var) " << &var<<endl;
    cout << "Hasil/value dari varPtr " << varPtr << endl;
    cout << "Hasil/value dari *varPtr " << *varPtr << endl;
    return 0;
}
```

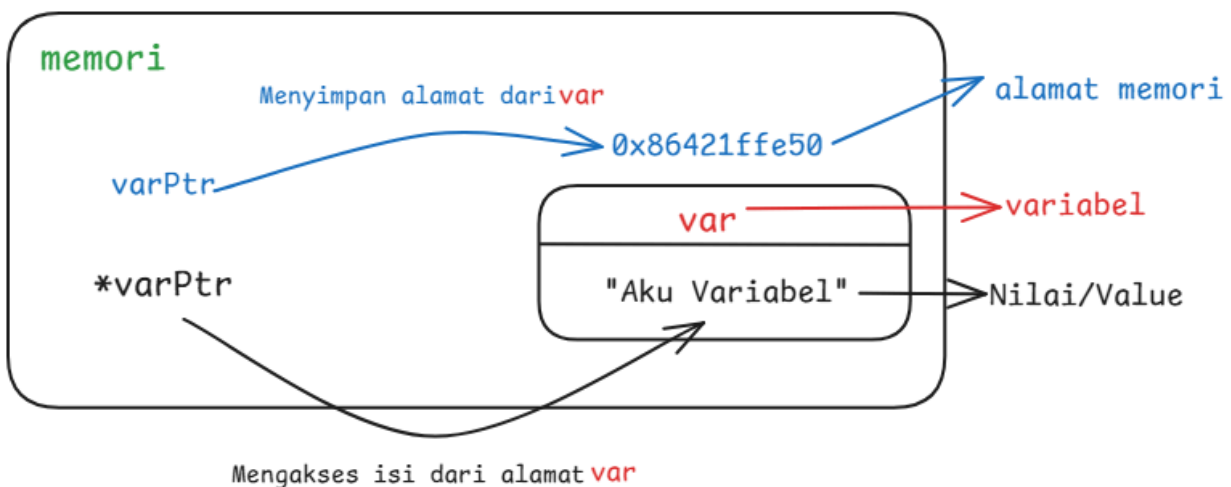



Output:

```
→ g++ buatPtr.cpp ; ./a.exe
Hasil dari varPtr: 0x86421ffe50
Hasil dari *varPtr: Aku Variabel

Kesimpulannya varPtr isi nya alamatnya var
*varPtr hasilnya value dari var
Jika masih bingung bisa amati output berikut

Hasil/value dari var Aku Variabel
Hasil/value dari alamat var (&var) 0x86421ffe50
Hasil/value dari varPtr 0x86421ffe50
Hasil/value dari *varPtr Aku Variabel
```



Pointer Pada Array

Pointer dapat juga digunakan untuk melakukan operasi pada array, di mana pointer akan menunjuk ke alamat memori baik sebagai array keseluruhan maupun untuk tiap nilai yang terdapat pada array. Contoh dapat dilihat pada potongan code berikut:



```
// arrayPtr.cpp
#include <iostream>
using namespace std;

int main () {
    cout << "pointer yang menunjuk ke suatu array"<< endl;
    int a[5] = {1,2,3,4,5};
    int (*aPtr)[5] = &a;
    for (int i =0; i <5; i++){
        cout << *aPtr <<endl;
    }

    cout << "Pointer yang menunjuk ke arah elemen array" <<endl;
    int b[5] = {1,2,3,4,5,};
    int *bPtr = b;
    for (int i = 0; i < 5; i++){
        cout << bPtr <<endl;
        // cout << *bPtr <<endl;
        bPtr++;
    }
    return 0;
}
```

Output:

```
→ g++ arrayPtr.cpp ; ./a.exe
pointer yang menunjuk ke suatu array
0x5ffe90
0x5ffe90
0x5ffe90
0x5ffe90
0x5ffe90
Pointer yang menunjuk ke arah elemen array
0x5ffe70
0x5ffe74
```



```
0x5ffe78  
0x5ffe7c  
0x5ffe80
```

Pointer Sebagai Parameter Fungsi

Pointer di function adalah konsep di mana kita menggunakan pointer sebagai parameter atau nilai kembalian (return value) dalam fungsi. Ini memungkinkan kita untuk mengirim alamat memori, menghemat memori, mengembalikan lebih dari satu nilai. Keuntungan menggunakan pointer di function adalah efisiensi memori karena dapat menghindari penyalinan data yang besar seperti struct atau array, dan dapat memodifikasi data asli.

Contohnya tanpa pointer kita punya fungsi untuk mengubah nilai variabel a:

```
// funcNoPtr.cpp  
#include <iostream>  
using namespace std;  
int ubahNilai(int a, int b){  
    return a=b;  
}  
int main () {  
    int a,b;  
    a=5;  
    b=20;  
    ubahNilai( a, b );  
    cout << a;  
    return 0;  
}
```

Output:

```
→ g++ funcNoPtr.cpp ; ./a.exe  
5
```



Seperti yang terlihat nilai variabel `a` tidak berubah, karena perubahan hanya terjadi di variabel lokal, variabel `a` di fungsi adalah duplikasi value dari variabel di main function. Untuk bisa mengakses variabel dari variabel aslinya kita menggunakan pointer di parameter. Untuk melakukan ini ada beberapa cara:

a. Penggunaan Address-of Operator (`&`) sebagai parameter fungsi

```
// funcPtr.cpp
#include <iostream>
using namespace std;
int ubahNilai(int &a, int b){
    return a=b;
}
int main () {
    int a,b;
    a=5;
    b=20;

    ubahNilai( a, b );
    cout << a;

    return 0;
}
```

Output:

```
→ g++ funcNoPtr.cpp ; ./a.exe
20
```



b. Penggunaan Dereference Operator (*) sebagai parameter fungsi

```
#include <iostream>
using namespace std;

int ubahNilai(int *a, int b){
    return *a=b;
}

int main () {
    int a,b;
    a=5;
    b=20;

    ubahNilai( &a, b );
    cout << a;

    return 0;
}
```

Output:

```
→ g++ funcNoPtr.cpp ; ./a.exe
20
```

Kedua cara tersebut memberi akses pada variabel asli di main.



Pointer Pada Struct

Pointer pada struct adalah konsep di mana kita menggunakan pointer untuk mengakses dan memanipulasi data yang ada di dalam sebuah struct. Struct adalah tipe data kustom yang menggabungkan beberapa variabel dengan tipe data yang berbeda, dan pointer memungkinkan kita untuk bekerja langsung dengan alamat memori dari struct tersebut.

Penggunaan pointer pada struct maupun class juga sangat penting untuk diketahui, karena konsep ini akan sangat berguna untuk memahami bagaimana tiap objek saling terkait, namun pada praktikum Algoritma dan Pemrograman Lanjut ini kita tidak akan membahasnya terlalu dalam dan akan lebih ditekankan pada praktikum Struktur Data. Namun tidak ada salahnya untuk mengenal bagaimana cara kerja pointer pada struct. Source code dapat kita lihat pada kode di bawah.

```
#include <iostream>
using namespace std;

struct Menu {
    string nama;
    float harga;
};

int main () {
    Menu nasgor;
    Menu *nasgorPtr = &nasgor;

    nasgor.nama = "Nasi Goreng Cumi Hitam Pak Kris";
    nasgor.harga = 15000;
    cout << nasgor.nama << " " << nasgor.harga<<endl;

    // kita bisa mengakses/manipulasi value variabel nasgor
    // melalui variabel nasgorPtr
    nasgorPtr->nama = "Nasi Goreng Mawut";
```



```
nasgorPtr->harga = 13000;  
cout << nasgor.nama << " " << nasgor.harga << endl;  
// kalau memberi value pada atribut biasa pakai ( . )  
// kalau memberi value pada atribut dari pointer pakai ( → )  
return 0;  
}
```

Berdasarkan code di atas, dapat dilihat bahwa terdapat perbedaan cara untuk mengakses atribut pada objek struct. Untuk memberi nilai pada atribut objek digunakan Dot Operator (.), namun hal ini dapat berubah apabila kita menggunakan pointer, untuk memberikan nilai pada atribut objek yang ditunjuk oleh pointer, diperlukan operator yang berbeda, yaitu Arrow Operator (->).



Studi Kasus

1. Mas Rizal sedang mengelola aplikasi dompet digital. Ia ingin membuat fitur otomatis di mana setiap kali saldo bertambah, sistem akan langsung memotong biaya administrasi sebesar Rp2.000. Mas Rizal ingin memastikan bahwa fungsi tersebut benar-benar memanipulasi saldo di alamat memori aslinya, bukan sekadar salinan data sementara.

Skenario :

- **Saldo Awal:** Rp100.000.
 - **Tugas:** Buatlah fungsi `updateSaldo` yang menerima **alamat memori** saldo tersebut. Di dalam fungsi, tambahkan saldo baru sebesar Rp50.000 dan langsung potong pajak Rp2.000. Tampilkan alamat memori saldo tersebut di dalam fungsi untuk membuktikan bahwa alamatnya tetap sama dengan yang ada di menu utama.
2. Juliro sedang mengatur antrean sembako di Shi Shan Lu. Ia mendapati ada dua nomor antrean yang tertukar posisinya dalam sistem. Juliro ingin menukar kedua nilai tersebut (swap) bukan dengan mengganti variabelnya secara manual, melainkan dengan cara "menembak" langsung ke alamat memori di mana nilai tersebut disimpan.

Skenario :

- **Antrean:** A = 105 dan B = 201.
- **Tugas:** Buatlah fungsi `tukarAntrean` yang menerima dua parameter berupa **Pointer**. Di dalam fungsi, tukar nilai yang tersimpan pada kedua alamat tersebut menggunakan **Dereference Operator**. Tampilkan hasil akhirnya di fungsi main untuk membuktikan bahwa nilai variabel aslinya sudah bertukar posisi.